

Belief-Score-Based POMDP 実験の関連研究と比較設計

Belief-Score-Based-Model 実験仕様

August 12, 2026

Abstract

本稿は、粒子上の score による信念更新、belief set から policy への作用素表現、連続行動の拡散方策、model-free POMDP reinforcement learning という四つの観点から関連研究を整理する。構成要素ごとの先行研究は存在し、とりわけ Zeng らの Conditional Score-based Filter は particle set、Set Transformer、conditional diffusion を同時に用いるため非常に近い。本実験の差分は、未知 POMDP において信念表現と方策を環境報酬から task-oriented に学習し、連続行動では per-denoising-step PPO を組み合わせる点にある。ただし PPO のみで学習した score を較正済み Bayesian posterior と呼ぶことはできない。この差分を検証するため、提案法、encoder ablation、Gaussian action ablation、model-free MLP/GRU、oracle-state 参照からなる 6 手法の比較実装も定義する。信念更新は保存 noise から現在 parameters で再生し、episode 全体へ逆伝播する full と環境時刻境界だけを切る tbptt_1 を比較する。active production では score Transformer と GRU へ同一の action head を接続し、GRU にも対応する二つの temporal-gradient mode を実装する。

1 今回の方法を構成する三つの写像

今回の方法は、概念的には次の三つの写像からなる。

1. 初期時刻には専用 energy $g_{\theta_0}(o_0, s) - \frac{1}{2}\|s\|^2$ を使い、それ以降は観測、前時刻の粒子、行動から新しい energy/score field を作る信念更新

$$\mathbf{s}_{t+1}(s') = \nabla_{s'} [f(o_{t+1}, s', a_t) + \text{logsumexp}_i u(s', s_{t,i}, a_t)] . \quad (1)$$

ここで二次項は parameter regularization ではなく、初期密度を拘束する標準 Gaussian 基準密度の log density である。無制限の関数族なら g_{θ_0} へ吸収できるが、有限幅 MLP が非有界な latent 空間全体で同じ二次 tail を正確に表す保証はないため、normalizable な裾と scale を明示する項として分離する。

2. $\{(s_{t,i}, \mathbf{s}_t(s_{t,i}))\}_{i=1}^K$ という順序なし集合から policy condition を作る set encoder。
3. condition と action query、または condition と noisy action を受け取り、離散 action logit または連続 action reverse kernel を返す operator policy。

以下では、この三要素のどこまでが各先行研究と一致するかを分けて述べる。

2 Score-based filtering と particle update

2.1 Score-based Filter

Bao, Zhang, Zhang の Score-based Filter[2] は、recursive Bayesian filtering へ score-based generative sampling を導入した代表的研究である。Bayes 則から posterior score を

$$\nabla_x \log p(x_t | y_{1:t}) = \nabla_x \log p(x_t | y_{1:t-1}) + \nabla_x \log p(y_t | x_t) \quad (2)$$

と分解し、学習した prediction score へ観測尤度 score を加えて reverse diffusion を行う。

一致点：密度そのものではなく score field を保持し、score-driven sampler で事後粒子を生成する点が一致する。式 (1) も、 f を log-likelihood、 u を log-transition として解釈すれば式 (2) の particle mixture 近似になる。

相違点：Score-based Filter は状態遷移モデルと観測尤度を既知として扱い、filtering 精度を目的に score を学習する。今回の f, u は未知 POMDP 内で環境報酬だけから学ばれるため、確率モデルとして同定される保証がない。

2.2 Score-based Sequential Langevin Sampling

Ding らの SSLS[3] は、予測粒子に denoising score matching を適用し、prediction score と likelihood gradient を足した drift で annealed Langevin Monte Carlo を行う。高次元・多峰 filtering に加えて収束および長時間誤差を解析している。

一致点：逐次予測、posterior score の加法分解、Langevin particle sampling の組合せが近い。

相違点：各時刻の score matching 用 sample と既知の尤度を必要とし、policy や control loss は持たない。今回の Unadjusted Langevin Algorithm は Metropolis 補正も annealing も行わない簡略形である。

2.3 Conditional Score-based Filter：最も近い filtering 研究

Zeng らの Conditional Score-based Filter (CSF) [4] は、prior particle ensemble を Set Transformer と Pooling by Multihead Attention で符号化し、その condition の下で diffusion model から posterior samples を生成する。offline で多様な prior/posterior pair を学習することにより、online 時刻ごとの再学習を避ける。

一致点：particle set、permutation-invariant Transformer pooling、conditional score/diffusion sampler を一つの pipeline で使う点は今回と直接重なる。したがって、新規性を論じる際の最重要引用である。

相違点：CSF は filtering/data assimilation 専用であり、既知モデルから作った教師 posterior を用いる。policy、PPO、action-conditioned belief update、task-oriented representation learning は扱わない。今回の新規性は「Set Transformer を score filter に使うこと」単独には置かず、POMDP 制御との end-to-end 統合に置くべきである。

2.4 Trajectory-level score assimilation

Rozet and Louppe[5] は、短い Markov segment から軌道 prior の score を学び、観測尤度で guide しながら全軌道を非自己回帰的に生成する。逐次 filter ではなく smoothing に近いが、prior score と likelihood score を分離できることの関連例である。

3 Set から関数を出力する operator architecture

3.1 Deep Sets と Set Transformer

Deep Sets[6] は、permutation-invariant な set function の基本形

$$F(\{z_i\}_{i=1}^K) = \rho\left(\sum_{i=1}^K \varphi(z_i)\right) \quad (3)$$

を与える。Set Transformer[7] は self-attention で要素間相互作用を表し、PMA で順序不変に pool する。今回の

$$z_i = [s_{t,i}, \mathbf{s}_t(s_{t,i})] \longrightarrow \text{TransformerEncoder} \longrightarrow \text{Pool} \quad (4)$$

は、位置符号化を持たない self-attention と順序不変 pooling からなる Set Transformer 型 encoder である。

本実験での位置付け：Deep Sets を architecture ablation、Transformer encoder を提案 encoder として同一 PPO 条件で比較する。性能差がなければ、提案の利点は attention ではなく score-based recurrent update にあると解釈すべきである。

3.2 Conditional Neural Processes と Attentive Neural Processes

Conditional Neural Processes (CNP) [8] は、context set $\{(x_i, y_i)\}$ を共有 encoder と順序不変 aggregation で要約し、target query x_* とともに decoder へ渡して $p(y_* | x_*, \{(x_i, y_i)\})$ を出力する。今回の対応は

$$(x_i, y_i, x_*) = (s_{t,i}, \mathfrak{s}_t(s_{t,i}), a) \quad (5)$$

であり、belief set から action-indexed policy function を返す構造に近い。

Attentive Neural Processes (ANP) [9] は context self-attention と target-to-context cross-attention を導入する。候補 action ごとに重要な belief particle が異なるなら、action query を belief tokens へ cross-attend させる ANP 型 decoder が自然である。

相違点: CNP/ANP は複数の教師付き function regression task から条件付き関数分布を学ぶ。今回の出力は RL policy であり、教師 target function は存在しない。また連続行動版では、action query は最終 action そのものではなく各拡散 step の (x^n, n) として denoiser へ入る。

3.3 DeepONet

DeepONet[10] は、入力関数を固定 sensor 上の値として branch network へ、出力座標を trunk network へ入力し、両者の組合せで operator value を返す。belief score function を入力、action を出力側座標と見れば、今回も

$$\mathfrak{s}_t(\cdot) \mapsto [a \mapsto \pi(a | \mathfrak{s}_t)] \quad (6)$$

という operator learning の形を持つ。

相違点: DeepONet は通常、固定 sensor と教師付き operator data を用いる。今回の particle 位置は毎時刻変化し、set encoder で処理され、学習信号は return のみである。したがって実装名称は「Set Transformer conditioned policy/operator」が中心で、DeepONet は概念的な operator 解釈として引用するのが適切である。

4 Particle belief を用いる POMDP reinforcement learning

4.1 DPFRL

Ma らの Discriminative Particle Filter Reinforcement Learning (DPFRL) [11] は、learned transition、observation compatibility、soft resampling からなる微分可能 particle filter を actor network へ組み込み、A2C で end-to-end 学習する。粒子は真の state posterior を再構成するためではなく、意思決定に有用な discriminative latent belief として学習される。

一致点: 未知 POMDP で particle set を recurrently 更新し、belief と policy を control objective から共同学習する点が今回に最も近い。PPO だけでは Bayesian calibration が保証されない今回の位置付けにも先例を与える。

相違点: DPFRL は importance weight と soft resampling を用い、belief は平均と moment generating function features で要約する。今回には明示的重み・resampling がなく、energy score + ULA と Set Transformer pooling を用いる。

4.2 DVRL

Deep Variational Reinforcement Learning (DVRL) [12] は、sequential latent generative model と SMC inference を actor-critic へ組み込み、RL loss と n -step ELBO で共同学習する。

一致点: 複数 particle で多峰な latent belief を維持し、policy と共同学習する。

相違点: DVRL は observation decoder を含む明示的生成モデルと variational lower bound を必要とする。今回の「完全モデルフリー」は観測再構成 loss を用いない discriminative training を意味する。元論文の optimizer は A2C であり、数値を PPO 実装と直接比較すべきではない。

4.3 FORBES

Flow-based Recurrent Belief State Learning (FORBES) [13] は、normalizing flow を variational recurrent model へ導入し、連続かつ多峰な belief density を表現する。得られた belief を downstream RL へ入力して visual-motor POMDP で評価する。

相違点：正規化可能な flow density と補助生成目的を用いるのに対し、今回の energy は正規化定数を計算せず、score と sample だけを利用する。完全再実装は比較コストが大きいため、主要な関連研究として引用し、実験比較は追加予算がある場合に限定する。

5 Recurrent model-free baseline

Ni, Eysenbach, Salakhutdinov[14] は、architecture と hyperparameter を丁寧に選んだ recurrent model-free RL が、21 個の POMDP 環境中 18 個で特殊化手法と同等以上になり得ることを示した。これは、複雑な belief 機構を提案する場合に GRU/LSTM baseline が必須であることを意味する。同論文の連続制御では主に recurrent SAC/TD3 を用いており、今回の GRU/LSTM-PPO は同じ主張に基づく apples-to-apples adaptation であって、原論文アルゴリズムの完全再現ではない。本実装の active GRU は、連続 task では提案法と同じ DiffusionPolicy、離散 task では同じ categorical head を使う。full は観測・直前行動の episode prefix を rollout 境界越しに current parameters で再生し、tbptt_1 は各環境 step へ入る hidden だけを detach する。したがって action family と forward 値を揃えたまま、記憶表現と時間方向の gradient 範囲を比較できる。

6 連続行動の diffusion policy optimization

DPPO[15] は denoising chain を内側 MDP とみなし、各 Gaussian reverse transition の尤度比へ PPO clipping を適用する。最終 action の周辺尤度や chain 全体の ratio 積は使用しない。

一致点：belief set embedding を通常の state condition の代わりに使えば、今回の連続行動方策へ直接適用できる。

相違点：DPPO の中心設定は behavior cloning 済み Diffusion Policy の fine-tuning であり、今回の全 network を環境報酬だけで学ぶ from-scratch 設定とは異なる。同論文の from-scratch 実験でも Gaussian policy より遅い。NCDPO[16] は denoising noise を condition として保存し、最終 action likelihood から全 denoising step へ backprop することで from-scratch 効率を改善する候補だが、現時点では preprint/submission である。

7 Model-free full-gradient belief replay

rollout 時には信念粒子座標を PPO 更新用の固定値として使わず、環境時刻 t の ULA 初期 noise z_t と各 step の noise $\epsilon_{t,1:L}$ を保存する。episode 初期時刻は専用 energy $g_{\eta_0}(o_0, s) - \frac{1}{2}\|s\|^2$ 、それ以降は観測 energy と前時刻粒子に対する遷移 energy の log-mean-exp を用いる。観測列 $o_{0:t}$ 、実現済み行動列 $a_{0:t-1}$ 、episode-start mask、報酬、old log-prob、advantage と return も保存 data として固定する。PPO 更新では現在の信念 parameters η を用いて

$$r_{t,0}^{\eta} = z_t, \quad (7)$$

$$r_{t,l}^{\eta} = r_{t,l-1}^{\eta} + \alpha_l \nabla_s E_{\eta}(r_{t,l-1}^{\eta}; o_t, B_{t-1}^{\eta}, a_{t-1}) + \sqrt{2\alpha_l} \epsilon_{t,l}, \quad (8)$$

$$B_t^{\eta} = \left\{ \left(r_{t,L,i}^{\eta}, \nabla_s E_{\eta}(r_{t,L,i}^{\eta}; o_t, B_{t-1}^{\eta}, a_{t-1}) \right) \right\}_{i=1}^K \quad (9)$$

を再 parameterize して再生する。全 ULA step と最後の score で計算 graph を保持するため、policy/value loss は粒子 trajectory、観測 energy、遷移 energy へ伝播する。環境の状態遷移や観測生成には微分せず、実現済み行動も固定 data として扱うので、既知環境モデルを用いない model-free 学習である。

full では、episode が rollout 境界を越えた場合も未終了 episode prefix の観測・行動・noise を保持し、episode-start から対象 loss 時刻まで current parameters で全 BPTT する。真の episode

reset だけが時間方向の graph 境界である。tbptt_1 では同一の forward 再生を行いつつ、各環境時刻の入力だけ

$$B_{t-1}^{\text{in}} = \text{stopgrad}(B_{t-1}^{\eta}) \quad (10)$$

とする。したがって時刻内の全 ULA step は両 mode で微分され、同じ noise に対する forward 値も一致する。

PPO では flatten した独立時刻 minibatch ではなく sequence minibatch を用い、各 optimizer step の current parameters で再生する。optimizer 更新後には未終了 episode prefix から belief context を再構築し、stale particle を次 rollout へ渡さない。full の activation memory を抑えるため non-reentrant checkpoint `torch.utils.checkpoint(..., use_reentrant=False)` を使う。これは backward 中に forward を再計算するだけで、detach を挿入せず数学的な勾配を変更しない。production では論理 minibatch 512 遷移を、環境 trajectory 軸だけで 4 個の 128 遷移 memory microbatch へ分ける。各 loss を論理 minibatch 内の遷移比で重み付けして gradient を加算し、gradient clipping と optimizer step は 4 回の backward 後に 1 回だけ行う。sequence を時間方向に分割しないため、この省 memory 化は full BPTT を TBPTT へ変えない。

8 実装した直接比較

8.1 Light-Dark の N 次元化

Light-Dark 原典 [1] は $x_t, u_t, o_t \in \mathbb{R}^2$, $x_{t+1} = x_t + u_t$ という零 process-noise の一次系であり、観測 noise は第 1 座標に依存する。本実装はこれを

$$x_t, u_t, o_t \in \mathbb{R}^N, \quad u_t \in [-1, 1]^N, \quad x_{t+1} = x_t + u_t, \quad (11)$$

$$o_t \sim \mathcal{N}\left(x_t, \left\{\frac{1}{2}(5 - x_{t,1})^2 + \sigma_{\min}^2\right\} I_N\right) \quad (12)$$

へ拡張する。初期状態は $\mathcal{N}(21, 0.5^2 I_N)$ 、目標は原点、step reward は $-(0.5\|x_t\|_2^2 + 0.5\|u_t\|_2^2)$ である。原典にない有界行動と目標半径 0.25 は、現在の Gym 型 PPO 実装へ合わせた変更である。設定の dimension で任意の N を指定でき、production では $N = 5$ を使う。

8.2 比較手法と実験行列

実行入口を `sb-pomdp-compare` に統一し、次の 6 手法を実装した。

method	信念・入力	action policy
<code>score_transformer</code>	score 粒子と Transformer (提案法)	categorical / diffusion
<code>score_deepsets</code>	同じ score 粒子と Deep Sets	categorical / diffusion
<code>score_gaussian</code>	score 粒子と Transformer	tanh Gaussian (連続のみ)
<code>observation_mlp</code>	現在の部分観測だけ	categorical / tanh Gaussian
<code>gru</code>	部分観測と直前行動, episode-prefix replay	categorical / diffusion (active 比較)
<code>oracle_state</code>	正規化した完全状態	categorical / tanh Gaussian

実装済み 6 手法を 4 task すべてで走らせる候補は 24 組だが、`score_gaussian` は離散 CartPole で提案法との差が定義できないため除外する。したがって実装全体の候補は 23 method/task 組であり、local 設定では 2 gradient modes を含む 46 cells を 1 seed・1 PPO update で smoke test する。これは成果物を含む実装経路の確認であり、性能比較ではない。active production は `score_transformer` と `gru` の 2 手法に限定し、4 tasks、2 gradient modes、各組 5 seeds、合計 $2 \times (2 \times 4) \times 5 = 80$ runs とする。active production で score belief を持たない baseline は `gru` だけである。GRU の full は episode 開始から hidden recurrence を rollout 境界越しに再生し、tbptt_1 は各環境 step の入力 hidden を detach する。両 mode は同一 forward 値と異なる temporal gradient 範囲を持つ実際の ablation である。

8.3 比較で固定する条件

全手法で task、train/evaluation seed、environment step budget、rollout 長、GAE、PPO clip、optimizer、終了処理、evaluation episode 数を共有する。active 比較の連続 3 task では、score Transformer と GRU へ同じ DiffusionPolicy を接続し、reverse step 数、noise schedule、variance floor、denoising advantage 重み、per-denoising-step PPO clipping を共有する。離散 CartPole でも同じ categorical head を使う。非 active な連続 Gaussian 系は同じ tanh-squashed Gaussian head と Jacobian 込み log-prob を使い、score_gaussian との比較では belief encoder も固定する。production では全手法共通で Adam の学習率を 5×10^{-5} 、PPO clip 幅を 0.2、gradient norm 上限を 0.5 とする。approximate KL と clip fraction は診断用に記録するが、KL 閾値による early stopping や minibatch 拒否は行わない。log ratio への独自 hard clamp も置かない。entropy bonus の係数は debug・production とともに 0 とし、比較目的を $-L_{\text{policy}} + c_V L_{\text{value}}$ へ揃える。全手法で effective minibatch 512 と optimizer 更新回数を共通にし、score 系の 4×128 という物理分割は更新回数に数えない。物理分割は throughput へ影響するため、wall-clock と peak GPU memory とともに明記する。GRU の PPO minibatch は固定長 time chunk へ分けず、完全な時間軸を保って環境軸だけを分割する。optimizer step 後は未終了 episode prefix を更新後 parameters で再生して online hidden を再構築する。評価は離散方策の greedy、diffusion の mean_chain、Gaussian の mean_action を決定論的規約とし、すべてで stochastic も記録する。ただし score 系では、決定論的なのは action readout だけであり、belief の初期粒子と ULA noise は sample する。したがって、これは agent 全体を決定論的にした評価ではない。

active production の集約表は gradient mode と 8 method/task 組ごとの 16 行を保ち、各行に deterministic action readout と stochastic 双方の seed 平均・標準偏差・同一 seed paired 差を別列で記録する。前者には greedy、mean_chain、mean_action という規約差が含まれるため、action family をまたぐ比較では stochastic 列を共通指標として重視する。これら三つは metric 名ではなく action readout 名である。active な連続比較は両手法とも mean_chain であり、決定論的 readout も揃う。事前に一つの primary metric を定めるなら、held-out episode における stochastic policy の最終 return を training seed 間で集約した値を用いる。粒子・拡散手法は 1 environment step 当たりの計算が大きいので、environment steps だけでなく wall-clock、peak GPU memory、parameter 数も報告する。active comparison の GRU は入力 encoder 幅 [348, 185]、hidden 幅 176 とし、score Transformer と同じ policy/value head の後段へ接続する。parameter 数は次の通りである。

task	score Transformer	GRU	相対差
CartPole	347,462	347,060	-0.116%
Pendulum	348,357	348,759	+0.115%
MountainCarContinuous	348,229	348,411	+0.052%
Light-Dark ($N = 5$)	351,817	352,223	+0.115%

最大絶対相対差は 0.116% である。共有 hyperparameter と capacity matching は比較条件を揃えるが、手法別に十分 tuning した比較ではない。local smoke の小型 network はこの capacity matching の対象外である。oracle_state も有限予算で最適化する参照法であり、理論的上限ではない。local smoke では score belief を持たない observation_mlp、gru、oracle_state も両 gradient mode で実行する。active production に含めるのはこのうち gru だけである。GRU では mode が temporal gradient 範囲を変える。一方、feed-forward な observation_mlp と oracle_state に対する mode label は network 計算を変えない local smoke 上の control 反復である。

学習時には、初期 energy g_0 、観測 energy f 、遷移 energy u の pre-clip gradient norm、初期時刻/再帰時刻別の score と particle の L2 norm、各区分の sample 数、score/particle の non-finite 数を記録する。これらは数値安定性と gradient flow の診断値であり、閾値による optimizer step 拒否や primary 性能指標には用いない。

8.4 今回の行列に含めない比較

DPFRL-style particle PPO、DVRL、FORBES、LSTM は未実装である。特に DPFRL の原論文は A2C であり、PPO へ移植した場合は原手法の再現ではなく adaptation と明記する必要がある。補助生成 loss を用いる DVRL/FORBES も、環境報酬だけを用いる今回の学習信号とは条件が異な

る。Light-Dark に 5 次元 Box 行動を追加するが、無限次元行動や多峰な最適行動分布まで網羅するものではなく、diffusion の一般的な優位性をこの比較だけでは検証できない。

9 新規性の境界と安全な主張

個々の構成要素について、score-based filter、Set Transformer pooling、particle belief RL、diffusion PPO はいずれも先行研究が存在する。特に CSF[4] との重なりから、「particle set を Transformer で条件付ける score filter」自体を新規とは主張できない。本実験で検証すべき組合せ上の差分は次である。

1. 未知の action-conditioned POMDP で、energy-defined score belief を逐次更新すること。
2. likelihood/transition 教師を使わず、policy return から task-oriented に belief と operator policy を共同学習すること。
3. 同じ belief set condition を離散 Softmax policy と連続 DDPM policy の双方へ接続すること。
4. 連続行動では per-denoising-step PPO を from scratch で適用し、同じ diffusion head を持つ GRU と比較すること。

一方、補助的な filtering loss や calibration test なしには、学習 score を「真の Bayesian posterior score」または「正確な belief」と断定しない。“task-oriented score-based latent belief” または “discriminative belief representation” が適切である。

References

- [1] Robert Platt Jr., Russ Tedrake, Leslie Kaelbling, Tomas Lozano-Perez, “Belief space planning assuming maximum likelihood observations,” Robotics: Science and Systems, 2010. Official: https://groups.csail.mit.edu/robotics-center/public_papers/Platt10.pdf.
- [2] Feng Bao, Zezhong Zhang, Guannan Zhang, “A score-based filter for nonlinear data assimilation,” *Journal of Computational Physics*, Vol. 514, 113207, 2024. DOI: <https://doi.org/10.1016/j.jcp.2024.113207>; arXiv: <https://arxiv.org/abs/2306.09282>.
- [3] Zhao Ding, Chenguang Duan, Yuling Jiao, Jerry Zhijian Yang, Cheng Yuan, Pingwen Zhang, “Nonlinear Assimilation with Score-based Sequential Langevin Sampling,” 2024. arXiv: <https://arxiv.org/abs/2411.13443>.
- [4] Zhijun Zeng, Weiye Gan, Junqing Chen, Zuoqiang Shi, “An Efficient Conditional Score-based Filter for High Dimensional Nonlinear Filtering Problems,” 2025. arXiv: <https://arxiv.org/abs/2509.19816>.
- [5] François Rozet, Gilles Louppe, “Score-based Data Assimilation,” NeurIPS, 2023. DOI: <https://doi.org/10.52202/075280-1763>; official: https://proceedings.neurips.cc/paper_files/paper/2023/hash/7f7fa581cc8a1970a4332920cdf87395-Abstract-Conference.html.
- [6] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, Alexander J. Smola, “Deep Sets,” NeurIPS, 2017. Official: <https://proceedings.neurips.cc/paper/2017/hash/f22e4747da1aa27e363d86d40ff442fe-Abstract.html>; arXiv: <https://arxiv.org/abs/1703.06114>.
- [7] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam Kosior, Seungjin Choi, Yee Whye Teh, “Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks,” ICML, 2019. Official: <https://proceedings.mlr.press/v97/lee19d.html>; arXiv: <https://arxiv.org/abs/1810.00825>.

- [8] Marta Garnelo, Dan Rosenbaum, Christopher Maddison, Tiago Ramalho, David Saxton, Murray Shanahan, Yee Whye Teh, Danilo Rezende, S. M. Ali Eslami, “Conditional Neural Processes,” ICML, 2018. Official: <https://proceedings.mlr.press/v80/garnelo18a.html>; arXiv: <https://arxiv.org/abs/1807.01613>.
- [9] Hyunjik Kim, Andriy Mnih, Jonathan Schwarz, Marta Garnelo, S. M. Ali Eslami, Dan Rosenbaum, Oriol Vinyals, Yee Whye Teh, “Attentive Neural Processes,” ICLR, 2019. arXiv: <https://arxiv.org/abs/1901.05761>.
- [10] Lu Lu, Pengzhan Jin, Guofei Pang, Zhongqiang Zhang, George Em Karniadakis, “Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators,” *Nature Machine Intelligence*, Vol. 3, pp. 218–229, 2021. DOI: <https://doi.org/10.1038/s42256-021-00302-5>.
- [11] Xiao Ma, Peter Karkus, David Hsu, Wee Sun Lee, Nan Ye, “Discriminative Particle Filter Reinforcement Learning for Complex Partial Observations,” ICLR, 2020. Official: https://openreview.net/forum?id=HJl8_eHYvS; arXiv: <https://arxiv.org/abs/2002.09884>.
- [12] Maximilian Igl, Luisa Zintgraf, Tuan Anh Le, Frank Wood, Shimon Whiteson, “Deep Variational Reinforcement Learning for POMDPs,” ICML, 2018. Official: <https://proceedings.mlr.press/v80/igl18a.html>; arXiv: <https://arxiv.org/abs/1806.02426>.
- [13] Xiaoyu Chen, Yao Mark Mu, Ping Luo, Shengbo Li, Jianyu Chen, “Flow-based Recurrent Belief State Learning for POMDPs,” ICML, 2022. Official: <https://proceedings.mlr.press/v162/chen22q.html>; arXiv: <https://arxiv.org/abs/2205.11051>.
- [14] Tianwei Ni, Benjamin Eysenbach, Ruslan Salakhutdinov, “Recurrent Model-Free RL Can Be a Strong Baseline for Many POMDPs,” ICML, 2022. Official: <https://proceedings.mlr.press/v162/ni22a.html>; arXiv: <https://arxiv.org/abs/2110.05038>.
- [15] Allen Z. Ren, Justin Lidard, Lars L. Ankile, Anthony Simeonov, Pulkit Agrawal, Anirudha Majumdar, Benjamin Burchfiel, Hongkai Dai, Max Simchowitz, “Diffusion Policy Policy Optimization,” ICLR, 2025. Official: https://proceedings.iclr.cc/paper_files/paper/2025/hash/c0749c39aaff9e9e4c91f7118bf21b1e-Abstract-Conference.html; arXiv: <https://arxiv.org/abs/2409.00588>; code: <https://github.com/irom-princeton/dppo>.
- [16] Ningyuan Yang, Jiaxuan Gao, Feng Gao, Yi Wu, Chao Yu, “Fine-tuning Diffusion Policies with Backpropagation Through Diffusion Timesteps,” 2025. arXiv: <https://arxiv.org/abs/2505.10482>.